Non-Functional Requirements (NFRs) — Retail Store Pricing Management System

1. Purpose

This document defines non-functional requirements for a web application that allows authorised users to upload pricing feeds, search pricing records, and edit and save records. The intended scale is approximately 3,000 stores across multiple countries.

Important: The target values below are proposed starting points for discussion and testing, not verified service commitments. Confirm them with business stakeholders and validate them through performance, security, and operational testing.

2. Performance

NFR-PERF-01: Search response time

- **Requirement:** Under an agreed representative load, common paginated search requests should return within 2 seconds at the 95th percentile.
- **How addressed:** Use pagination, query-specific indexes, efficient SQL, connection pooling, and bounded response sizes.
- **How verified:** Run load tests using representative pricing data and common filter combinations; record p50, p95, p99 latency and error rate.

NFR-PERF-02: Record update response time

- **Requirement:** A normal single-record save should complete within 2 seconds at the 95th percentile under the agreed load.
- **How addressed:** Validate input, keep transactions short, and return only necessary data.
- **How verified:** Measure end-to-end save latency and confirm the updated value is persisted.

NFR-PERF-03: CSV upload processing

- **Requirement:** Define a maximum upload size and acceptable processing time after sample feed sizes and upload frequency are confirmed.
- **How addressed:** Validate file type and headers, stream or process bounded batches where appropriate, and avoid unbounded memory use.
- **How verified:** Test valid and invalid files at expected and maximum agreed sizes; measure processing time, memory use, and database load.

3. Scalability

NFR-SCALE-01: Multi-store capacity

- **Requirement:** Support data from approximately 3,000 stores across multiple countries. Required record volume and concurrency must be established separately.
- **How addressed:** Use stateless API instances where practical, pagination, appropriate indexes, connection pooling, and database capacity monitoring.
- **How verified:** Run load and stress tests with representative store counts, record volumes, feed sizes, and concurrent users.

NFR-SCALE-02: Growth

- **Requirement:** The design should permit the API tier to scale without requiring changes to the frontend's business workflow.
- **How addressed:** Keep API instances stateless where practical and place them behind a reverse proxy or load balancer if deployment needs it.
- **How verified:** Demonstrate deployment of multiple API instances and test session-independent request handling.

4. Availability and Reliability

NFR-AVAIL-01: Availability target

- **Requirement:** Agree a service availability target with stakeholders before production deployment. No percentage is assumed in this document.
- **How addressed:** Use health checks, deployment monitoring, appropriate redundancy, and documented operational procedures based on the agreed target.
- **How verified:** Monitor service uptime and review incidents against the agreed measurement window.

NFR-REL-01: Consistent data changes

- **Requirement:** A successful save must be persisted; a failed transaction must not leave a partially applied change within that transaction.
- **How addressed:** Use database transactions, input validation, constraints, and clear API error handling.
- **How verified:** Test successful saves, validation failures, database errors, and transaction rollback behaviour.

NFR-REL-02: Import error handling

- **Requirement:** Users must receive an understandable summary of CSV processing outcomes, including rejected rows and reasons where supported by the agreed import policy.

- **How addressed:** Define an explicit all-or-nothing or batch import policy and return structured validation details.
- **How verified:** Test missing columns, invalid prices, invalid dates, duplicate rows, and mixed-validity files.

5. Security and Privacy

NFR-SEC-01: Authentication and authorisation

- **Requirement:** Only authenticated users may access protected functions, and upload/edit actions must be restricted to authorised roles.
- **How addressed:** Integrate an agreed identity provider and enforce role checks in FastAPI. Do not rely only on frontend route protection.
- **How verified:** Test unauthenticated requests, permitted roles, and forbidden upload/edit actions.

NFR-SEC-02: Transport and secret protection

- **Requirement:** Use HTTPS in deployed environments and do not store credentials or secrets in source control or frontend code.
- **How addressed:** Configure TLS at the hosting edge, use environment-specific secret management, and restrict database network access.
- **How verified:** Review deployment configuration and repository contents; test that database access is not exposed to browsers.

NFR-SEC-03: Input and file validation

- **Requirement:** Reject unsupported files and invalid input safely.
- **How addressed:** Validate content type, file size, required CSV headers, field formats, and business rules on the backend; use parameterised database access.
- **How verified:** Test malformed CSVs, oversized uploads, unexpected fields, invalid identifiers, and common injection payloads.

NFR-SEC-04: Least privilege

- **Requirement:** Each application component and user should receive only the access required for its responsibilities.
- **How addressed:** Use restricted database credentials, role-based access, and private network connectivity to the database.
- **How verified:** Review role permissions and attempt unauthorised database and API operations.

6. Data Integrity and Auditability

NFR-DATA-01: Valid pricing data

- **Requirement:** Persisted records must conform to agreed rules for Store ID, SKU, Product Name, Price, and Date.
- **How addressed:** Use suitable database types, required-field constraints, backend validation, and agreed uniqueness rules.
- **How verified:** Test boundary values, missing values, duplicate combinations, and invalid formats.

NFR-DATA-02: Price precision and currency

- **Requirement:** Price must use fixed-precision numeric storage. Currency handling must be agreed for multi-country use.
- **How addressed:** Use a decimal database type and add an explicit currency field if the business rules require it.
- **How verified:** Test decimal values and rounding rules against agreed examples.

NFR-DATA-03: Change history

- **Requirement:** Confirm whether the business requires a history of who changed a price, when it changed, and the previous and new values.
- **How addressed:** If required, add an audit table or equivalent controlled audit mechanism with user attribution and timestamps.
- **How verified:** Confirm edits create the expected audit entries and that ordinary users cannot alter audit history.

7. Maintainability and Testability

NFR-MAINT-01: Separation of concerns

- **Requirement:** Keep UI, API/business logic, and persistence responsibilities separated.
- **How addressed:** Use React components, FastAPI route/service modules, and a dedicated data-access layer or clearly structured database operations.
- **How verified:** Review the code structure and test critical API and validation logic independently.

NFR-MAINT-02: API documentation

- **Requirement:** Document API endpoints, request fields, response formats, and error cases.
- **How addressed:** Use FastAPI's generated API documentation and keep endpoint contracts aligned with the frontend.
- **How verified:** Review API documentation against implemented endpoints and automated API tests.

NFR-MAINT-03: Automated testing

- **Requirement:** Provide tests for core upload, search, edit, validation, and error-handling workflows.
- **How addressed:** Add unit tests for validation and business rules, API integration tests, and frontend workflow tests.
- **How verified:** Run the test suite in the development or CI pipeline and review coverage of critical flows.

8. Monitoring, Backup, and Recovery

NFR-OPS-01: Monitoring and logging

- **Requirement:** Operational errors and service health must be observable without logging secrets.
- **How addressed:** Capture structured application logs, health checks, request latency, error rates, upload failures, and database resource usage.
- **How verified:** Trigger representative errors and confirm they can be found and investigated.

NFR-OPS-02: Backup and restore

- **Requirement:** Define backup frequency, retention, Recovery Point Objective (RPO), and Recovery Time Objective (RTO) with stakeholders.
- **How addressed:** Configure database backups and document restoration steps.
- **How verified:** Perform and record restore exercises; confirm recovered data and recovery time against agreed objectives.

9. Usability and Compatibility

NFR-UX-01: Clear feedback

- **Requirement:** Users should understand upload status, validation failures, search results, and save outcomes.
- **How addressed:** Display clear status messages, field-level validation, and actionable error details without exposing internal stack traces.
- **How verified:** Test common success and failure flows with representative users or acceptance scenarios.

NFR-COMP-01: Browser support

- **Requirement:** Agree the supported browsers and versions.
- **How addressed:** Use supported React/browser features and test the agreed browser matrix.
- **How verified:** Run smoke and workflow tests in each supported browser.

10. Open Questions to Finalise

1. What are the expected concurrent users and daily upload frequency?
2. What is the typical and maximum CSV size or row count?
3. What search response time and availability targets are required?
4. What are the required RPO and RTO?
5. Which authentication provider and user roles are required?
6. What currency and date rules apply across countries?
7. How should duplicates and invalid rows be handled?
8. Is a price-change audit history mandatory?
9. Which browsers and hosting environment must be supported?

11. Acceptance Approach

Before release, agree measurable targets with stakeholders and execute performance, security, functional, recovery, and compatibility tests. The proposed numerical response-time targets in this document should remain provisional until the workload profile and acceptance criteria are approved.